

Cryptography

CPA Security and Pseudorandom Functions

Arkady Yerukhimovich

September 14, 2026

Announcements

- Homework 2 is out – due September 21 before class
- Scheduling update:
 - Final Exam: Monday, December 7
 - Project Workshop: Wednesday, December 9
- Suggested project topics will be posted on Piazza soon

Outline

- 1 Lecture 2 Review
- 2 Proofs by Reduction
- 3 Chosen-Plaintext Attack Security
- 4 Pseudorandom Functions
- 5 CPA-Secure Encryption from a PRF

Where Are We Now?

- PRG+OTP
 - Short key, long message
 - Security against an eavesdropper

Where Are We Now?

- PRG+OTP
 - Short key, long message
 - Security against an eavesdropper
- Limitations
 - One encryption per key
 - Reusing the mask reveals $m_1 \oplus m_2$

Where Are We Now?

- PRG+OTP
 - Short key, long message
 - Security against an eavesdropper
- Limitations
 - One encryption per key
 - Reusing the mask reveals $m_1 \oplus m_2$
- Today
 - Many encryptions per key
 - Adversarially chosen messages

Outline

- 1 Lecture 2 Review
- 2 Proofs by Reduction**
- 3 Chosen-Plaintext Attack Security
- 4 Pseudorandom Functions
- 5 CPA-Secure Encryption from a PRF

Reduction Refresher

To prove construction C secure using assumption P :

- 1 Assume PPT $\mathcal{A}_C$ breaks C

To prove construction C secure using assumption P :

- 1 Assume PPT $\mathcal{A}_c$ breaks C
- 2 Construct PPT $\mathcal{A}_r$ attacking P

To prove construction C secure using assumption P :

- 1 Assume PPT $\mathcal{A}_C$ breaks C
- 2 Construct PPT $\mathcal{A}_r$ attacking P
- 3 $\mathcal{A}_r$ runs $\mathcal{A}_C$ as a subroutine

To prove construction C secure using assumption P :

- 1 Assume PPT $\mathcal{A}_c$ breaks C
- 2 Construct PPT $\mathcal{A}_r$ attacking P
- 3 $\mathcal{A}_r$ runs $\mathcal{A}_c$ as a subroutine
- 4 Simulate $\mathcal{A}_c$'s security game

To prove construction C secure using assumption P :

- 1 Assume PPT $\mathcal{A}_c$ breaks C
- 2 Construct PPT $\mathcal{A}_r$ attacking P
- 3 $\mathcal{A}_r$ runs $\mathcal{A}_c$ as a subroutine
- 4 Simulate $\mathcal{A}_c$'s security game
- 5 Relate the success probabilities

To prove construction C secure using assumption P :

- 1 Assume PPT $\mathcal{A}_c$ breaks C
- 2 Construct PPT $\mathcal{A}_r$ attacking P
- 3 $\mathcal{A}_r$ runs $\mathcal{A}_c$ as a subroutine
- 4 Simulate $\mathcal{A}_c$'s security game
- 5 Relate the success probabilities
- 6 Contradiction

Exercise 1: PRGs

Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ be a secure PRG. Define

$$G'(s) = \overline{G(s)}.$$

Prove that G' is a secure PRG.

Exercise 1: PRGs

Let $G : \{0, 1\}^n \rightarrow \{0, 1\}^{n+1}$ be a secure PRG. Define

$$G'(s) = \overline{G(s)}.$$

Prove that G' is a secure PRG. Guiding questions:

- 1 What do we assume about $\mathcal{A}_c$?
- 2 What game is $\mathcal{A}_r$ playing?
- 3 How does $\mathcal{A}_r$ use its challenge and $\mathcal{A}_c$?
- 4 Why does $\mathcal{A}_r$ succeed?

5 minutes

Exercise 1: Reduction

Given challenge $w \in \{0, 1\}^{n+1}$:

- 1 $\mathcal{A}_r$ computes $w' = \overline{w}$

Exercise 1: Reduction

Given challenge $w \in \{0, 1\}^{n+1}$:

- 1 $\mathcal{A}_r$ computes $w' = \overline{w}$
- 2 $\mathcal{A}_r$ gives w' to $\mathcal{A}_c$

Exercise 1: Reduction

Given challenge $w \in \{0, 1\}^{n+1}$:

- 1 $\mathcal{A}_r$ computes $w' = \overline{w}$
- 2 $\mathcal{A}_r$ gives w' to $\mathcal{A}_c$
- 3 $\mathcal{A}_r$ outputs $\mathcal{A}_c$'s output

Exercise 1: Reduction

Given challenge $w \in \{0, 1\}^{n+1}$:

- 1 $\mathcal{A}_r$ computes $w' = \overline{w}$
- 2 $\mathcal{A}_r$ gives w' to $\mathcal{A}_c$
- 3 $\mathcal{A}_r$ outputs $\mathcal{A}_c$'s output

Analysis:

- $w = G(s) \Rightarrow w' = G'(s)$

Exercise 1: Reduction

Given challenge $w \in \{0, 1\}^{n+1}$:

- 1 $\mathcal{A}_r$ computes $w' = \overline{w}$
- 2 $\mathcal{A}_r$ gives w' to $\mathcal{A}_c$
- 3 $\mathcal{A}_r$ outputs $\mathcal{A}_c$'s output

Analysis:

- $w = G(s) \Rightarrow w' = G'(s)$
- $w \leftarrow \{0, 1\}^{n+1} \Rightarrow w'$ uniform

Exercise 1: Reduction

Given challenge $w \in \{0, 1\}^{n+1}$:

- 1 $\mathcal{A}_r$ computes $w' = \overline{w}$
- 2 $\mathcal{A}_r$ gives w' to $\mathcal{A}_c$
- 3 $\mathcal{A}_r$ outputs $\mathcal{A}_c$'s output

Analysis:

- $w = G(s) \Rightarrow w' = G'(s)$
- $w \leftarrow \{0, 1\}^{n+1} \Rightarrow w'$ uniform
- Same distinguishing advantage

Exercise 2: Encryption

Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ be secure against an eavesdropper. Define

$$\text{Enc}'_k(m) = \overline{\text{Enc}_k(m)}.$$

Prove that Π' is secure against an eavesdropper.

This one you will do on your own – 10 minutes.

Exercise 2: Reduction

- $\text{Dec}'_k(c') = \text{Dec}_k(\overline{c'})$

Exercise 2: Reduction

- $\text{Dec}'_k(c') = \text{Dec}_k(\overline{c'})$
- $\mathcal{A}_r$ runs $\mathcal{A}_c$ to obtain m_0, m_1

Exercise 2: Reduction

- $\text{Dec}'_k(c') = \text{Dec}_k(\overline{c'})$
- $\mathcal{A}_r$ runs $\mathcal{A}_c$ to obtain m_0, m_1
- $\mathcal{A}_r$ forwards m_0, m_1 to its challenger

Exercise 2: Reduction

- $\text{Dec}'_k(c') = \text{Dec}_k(\overline{c'})$
- $\mathcal{A}_r$ runs $\mathcal{A}_c$ to obtain m_0, m_1
- $\mathcal{A}_r$ forwards m_0, m_1 to its challenger
- $\mathcal{A}_r$ receives $c = \text{Enc}_k(m_b)$

Exercise 2: Reduction

- $\text{Dec}'_k(c') = \text{Dec}_k(\overline{c'})$
- $\mathcal{A}_r$ runs $\mathcal{A}_c$ to obtain m_0, m_1
- $\mathcal{A}_r$ forwards m_0, m_1 to its challenger
- $\mathcal{A}_r$ receives $c = \text{Enc}_k(m_b)$
- $\mathcal{A}_r$ gives $c' = \overline{c}$ to $\mathcal{A}_c$

Exercise 2: Reduction

- $\text{Dec}'_k(c') = \text{Dec}_k(\overline{c'})$
- $\mathcal{A}_r$ runs $\mathcal{A}_c$ to obtain m_0, m_1
- $\mathcal{A}_r$ forwards m_0, m_1 to its challenger
- $\mathcal{A}_r$ receives $c = \text{Enc}_k(m_b)$
- $\mathcal{A}_r$ gives $c' = \overline{c}$ to $\mathcal{A}_c$
- $\mathcal{A}_r$ outputs $\mathcal{A}_c$'s guess

Exercise 2: Reduction

- $\text{Dec}'_k(c') = \text{Dec}_k(\overline{c'})$
- $\mathcal{A}_r$ runs $\mathcal{A}_c$ to obtain m_0, m_1
- $\mathcal{A}_r$ forwards m_0, m_1 to its challenger
- $\mathcal{A}_r$ receives $c = \text{Enc}_k(m_b)$
- $\mathcal{A}_r$ gives $c' = \overline{c}$ to $\mathcal{A}_c$
- $\mathcal{A}_r$ outputs $\mathcal{A}_c$'s guess

Analysis

Perfect simulation: $c' = \text{Enc}'_k(m_b)$. Same success probability.

Reduction Checklist

- What does $\mathcal{A}_r$ receive?

Reduction Checklist

- What does $\mathcal{A}_r$ receive?
- What must $\mathcal{A}_c$ see?

Reduction Checklist

- What does $\mathcal{A}_r$ receive?
- What must $\mathcal{A}_c$ see?
- Can $\mathcal{A}_r$ simulate it without the secret?

Reduction Checklist

- What does $\mathcal{A}_r$ receive?
- What must $\mathcal{A}_c$ see?
- Can $\mathcal{A}_r$ simulate it without the secret?
- What happens in each challenger case?

Reduction Checklist

- What does $\mathcal{A}_r$ receive?
- What must $\mathcal{A}_c$ see?
- Can $\mathcal{A}_r$ simulate it without the secret?
- What happens in each challenger case?
- How do the success probabilities relate?

Outline

- 1 Lecture 2 Review
- 2 Proofs by Reduction
- 3 Chosen-Plaintext Attack Security**
- 4 Pseudorandom Functions
- 5 CPA-Secure Encryption from a PRF

Chosen-Plaintext Attack (CPA)

- $\mathcal{A}$: requests encryptions of messages of its choice
- Goal: learn which of two challenge messages was encrypted

Chosen-Plaintext Attack (CPA)

- $\mathcal{A}$: requests encryptions of messages of its choice
- Goal: learn which of two challenge messages was encrypted

Real-World Examples:

- German messages reporting locations of British mines
- “AF is low on water” at the Battle of Midway

Oracles in Cryptography

- Oracle: evaluates a function without revealing its internals

Oracles in Cryptography

- Oracle: evaluates a function without revealing its internals
- Example: $\text{Enc}_k(\cdot)$
 - Choose m
 - Receive $\text{Enc}_k(m)$
 - Do not learn k

Oracles in Cryptography

- Oracle: evaluates a function without revealing its internals
- Example: $\text{Enc}_k(\cdot)$
 - Choose m
 - Receive $\text{Enc}_k(m)$
 - Do not learn k
- Notation: $\mathcal{A}^{\mathcal{O}(\cdot)}$

Oracles in Cryptography

- Oracle: evaluates a function without revealing its internals
- Example: $\text{Enc}_k(\cdot)$
 - Choose m
 - Receive $\text{Enc}_k(m)$
 - Do not learn k
- Notation: $\mathcal{A}^{\mathcal{O}(\cdot)}$
- Each oracle call: one computation step

Defining CPA-Secure Encryption

$\text{PrivK}_{\mathcal{A}, \Pi}^{cpa}(n)$

Defining CPA-Secure Encryption

$\text{PrivK}_{\mathcal{A}, \Pi}^{cpa}(n)$

- Challenger: $k \leftarrow \text{Gen}(1^n)$

Defining CPA-Secure Encryption

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$

- Challenger: $k \leftarrow \text{Gen}(1^n)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: m_0, m_1 with $|m_0| = |m_1|$

Defining CPA-Secure Encryption

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$

- Challenger: $k \leftarrow \text{Gen}(1^n)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: m_0, m_1 with $|m_0| = |m_1|$
- Challenger: $b \leftarrow \{0, 1\}$; $c \leftarrow \text{Enc}_k(m_b)$

Defining CPA-Secure Encryption

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$

- Challenger: $k \leftarrow \text{Gen}(1^n)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: m_0, m_1 with $|m_0| = |m_1|$
- Challenger: $b \leftarrow \{0, 1\}$; $c \leftarrow \text{Enc}_k(m_b)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: guess b'

Defining CPA-Secure Encryption

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$

- Challenger: $k \leftarrow \text{Gen}(1^n)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: m_0, m_1 with $|m_0| = |m_1|$
- Challenger: $b \leftarrow \{0, 1\}$; $c \leftarrow \text{Enc}_k(m_b)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: guess b'
- Output 1 iff $b' = b$

Defining CPA-Secure Encryption

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$

- Challenger: $k \leftarrow \text{Gen}(1^n)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: m_0, m_1 with $|m_0| = |m_1|$
- Challenger: $b \leftarrow \{0, 1\}$; $c \leftarrow \text{Enc}_k(m_b)$
- $\mathcal{A}^{\text{Enc}_k(\cdot)}$: guess b'
- Output 1 iff $b' = b$

Definition

Π is CPA-secure if, for every PPT $\mathcal{A}$,

$$\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n).$$

The CPA Security Game

Challenger

Adversary $\mathcal{A}$

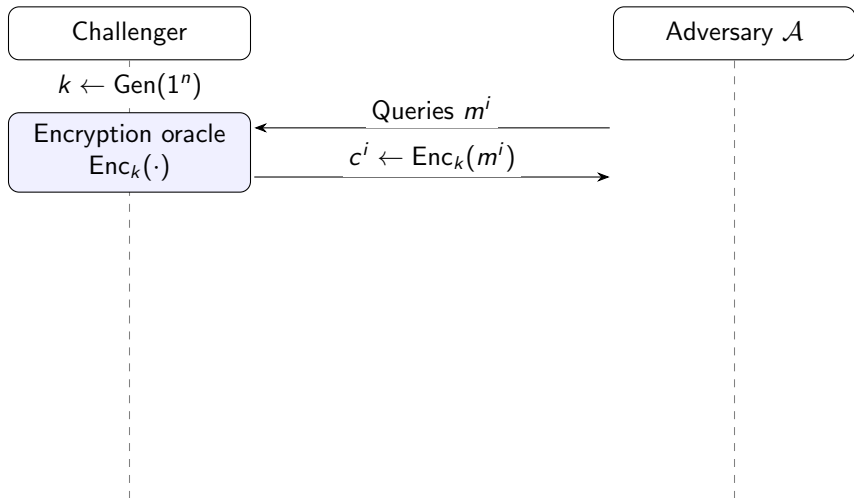
The CPA Security Game

Challenger

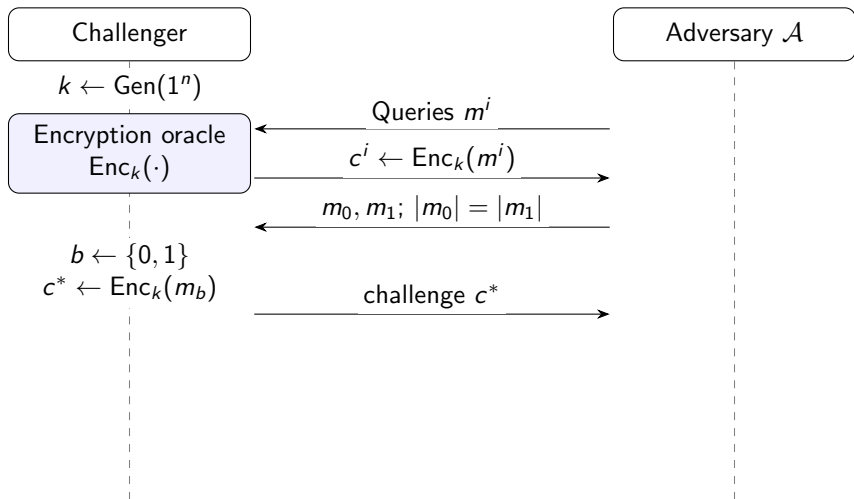
$k \leftarrow \text{Gen}(1^n)$

Adversary $\mathcal{A}$

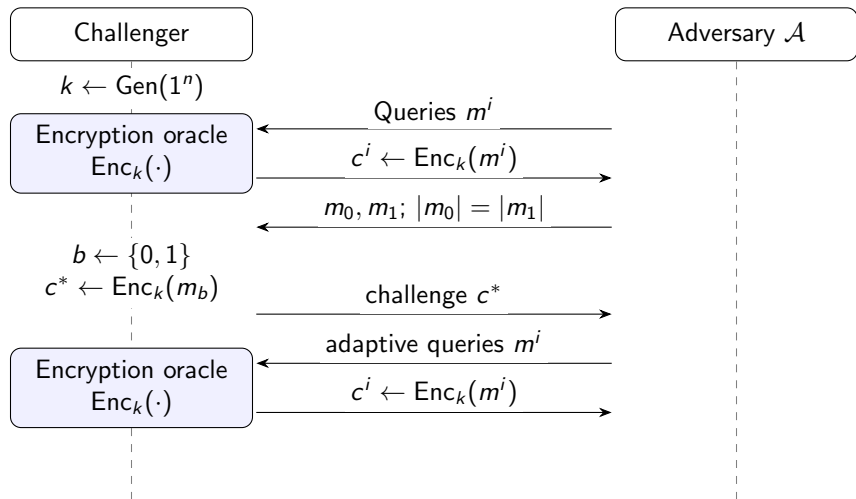
The CPA Security Game



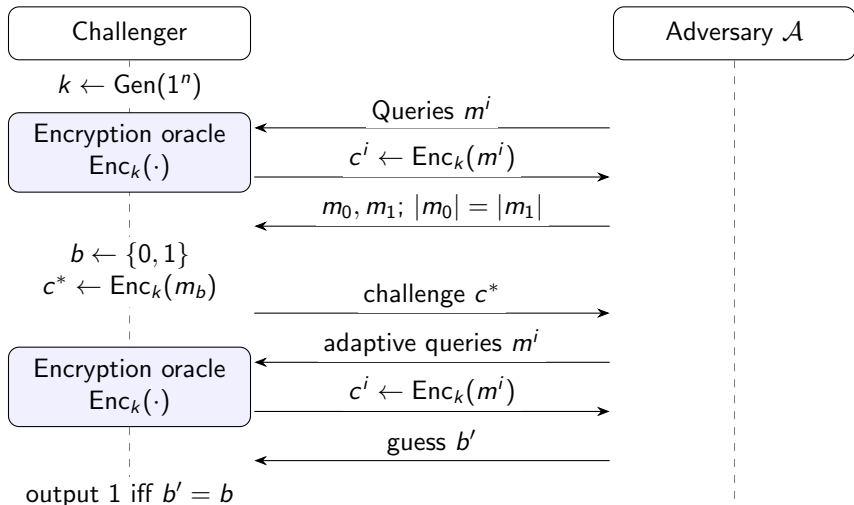
The CPA Security Game



The CPA Security Game



The CPA Security Game



- Encryption oracle queries before *and* after the challenge

Observations

- Encryption oracle queries before *and* after the challenge
- Queries may include m_0 and m_1 (the two challenge messages)

- Encryption oracle queries before *and* after the challenge
- Queries may include m_0 and m_1 (the two challenge messages)
- In CPA-secure encryption, $\text{Enc}_k(\cdot)$ must be randomized

Benefits of CPA Security

- Can encrypt many messages
 - $\mathcal{A}$ gets to see encryptions of many messages of its choice, still cannot break security of challenge
 - Can show that this means that seeing many ciphertexts doesn't help break any of them

Benefits of CPA Security

- Can encrypt many messages
 - $\mathcal{A}$ gets to see encryptions of many messages of its choice, still cannot break security of challenge
 - Can show that this means that seeing many ciphertexts doesn't help break any of them
- Can encrypt arbitrarily long messages
 - Break message m into n -bit blocks, $m = m_1 || m_2 || \dots || m_\ell$
 - To encrypt m separately encrypt each m_i .
 - Secure since this is just encrypting many messages

How to Construct CPA-Secure Encryption

- Recall that PRG+OTP encryption allowed us to encrypt long messages.
- But, it still revealed if same message was encrypted many times.

How to Construct CPA-Secure Encryption

- Recall that PRG+OTP encryption allowed us to encrypt long messages.
- But, it still revealed if same message was encrypted many times.

Key Idea

What if encryption (and decryption) could generate a fresh random mask for each ciphertext?

How to Construct CPA-Secure Encryption

- Recall that PRG+OTP encryption allowed us to encrypt long messages.
- But, it still revealed if same message was encrypted many times.

Key Idea

What if encryption (and decryption) could generate a fresh random mask for each ciphertext?

Note: We need to produce enough OTP's for as many encryptions as $\mathcal{A}$ wants. So, can't just pre-generate them all.

10-minute break

Outline

- 1 Lecture 2 Review
- 2 Proofs by Reduction
- 3 Chosen-Plaintext Attack Security
- 4 Pseudorandom Functions**
- 5 CPA-Secure Encryption from a PRF

What Is a Random Function?

For $f : \{0,1\}^n \rightarrow \{0,1\}^n$:

x	$f(x)$
0000	
0001	
$\vdots$	
1111	

- Each $f(x)$ chosen independently and uniformly

What Is a Random Function?

For $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$:

x	$f(x)$
0000	1010
0001	
$\vdots$	
1111	

- Each $f(x)$ chosen independently and uniformly
- Same input $\Rightarrow$ same output
- Fresh input $\Rightarrow$ fresh uniform output

What Is a Random Function?

For $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$:

x	$f(x)$
0000	1010
0001	0001
$\vdots$	
1111	

- Each $f(x)$ chosen independently and uniformly
- Same input $\Rightarrow$ same output
- Fresh input $\Rightarrow$ fresh uniform output

What Is a Random Function?

For $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$:

x	$f(x)$
0000	1010
0001	0001
$\vdots$	$\vdots$
1111	1101

- Each $f(x)$ chosen independently and uniformly
- Same input $\Rightarrow$ same output
- Fresh input $\Rightarrow$ fresh uniform output

Random Functions for Cryptography

- 2^n potential OTPs

Random Functions for Cryptography

- 2^n potential OTPs
- Fresh x : $f(x)$ uniformly random

Random Functions for Cryptography

- 2^n potential OTPs
- Fresh x : $f(x)$ uniformly random
- Problem: no efficient representation

Random Functions for Cryptography

- 2^n potential OTPs
- Fresh x : $f(x)$ uniformly random
- Problem: no efficient representation
 - 2^n table entries

Random Functions for Cryptography

- 2^n potential OTPs
- Fresh x : $f(x)$ uniformly random
- Problem: no efficient representation
 - 2^n table entries
 - $n2^n$ bits

Random Functions for Cryptography

- 2^n potential OTPs
- Fresh x : $f(x)$ uniformly random
- Problem: no efficient representation
 - 2^n table entries
 - $n2^n$ bits
 - 2^{n2^n} possible functions

Random Functions for Cryptography

- 2^n potential OTPs
- Fresh x : $f(x)$ uniformly random
- Problem: no efficient representation
 - 2^n table entries
 - $n2^n$ bits
 - 2^{n2^n} possible functions

Goal

Benefits of a random function, but with an efficient key and evaluation.

Deterministic, efficiently computable keyed function

$$F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$$

such that:

- $F_k(\cdot)$ efficiently computable

Deterministic, efficiently computable keyed function

$$F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$$

such that:

- $F_k(\cdot)$ efficiently computable
- Random k : $F_k(\cdot)$ looks like a random function

Deterministic, efficiently computable keyed function

$$F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$$

such that:

- $F_k(\cdot)$ efficiently computable
- Random k : $F_k(\cdot)$ looks like a random function
- Adversary does not know k

$\text{PRF}_{\mathcal{D},F}(n)$

$\text{PRF}_{\mathcal{D},F}(n)$

- Challenger chooses $b \leftarrow \{0, 1\}$

$\text{PRF}_{\mathcal{D},F}(n)$

- Challenger chooses $b \leftarrow \{0, 1\}$
- If $b = 0$: $f \leftarrow \mathcal{F}_n$; oracle $\mathcal{O} = f$

$\text{PRF}_{\mathcal{D},F}(n)$

- Challenger chooses $b \leftarrow \{0, 1\}$
- If $b = 0$: $f \leftarrow \mathcal{F}_n$; oracle $\mathcal{O} = f$
- If $b = 1$: $k \leftarrow \{0, 1\}^n$; oracle $\mathcal{O} = F_k$

$\text{PRF}_{\mathcal{D},F}(n)$

- Challenger chooses $b \leftarrow \{0, 1\}$
- If $b = 0$: $f \leftarrow \mathcal{F}_n$; oracle $\mathcal{O} = f$
- If $b = 1$: $k \leftarrow \{0, 1\}^n$; oracle $\mathcal{O} = F_k$
- $\mathcal{D}^{\mathcal{O}(\cdot)}$ outputs b'

$\text{PRF}_{\mathcal{D},F}(n)$

- Challenger chooses $b \leftarrow \{0, 1\}$
- If $b = 0$: $f \leftarrow \mathcal{F}_n$; oracle $\mathcal{O} = f$
- If $b = 1$: $k \leftarrow \{0, 1\}^n$; oracle $\mathcal{O} = F_k$
- $\mathcal{D}^{\mathcal{O}(\cdot)}$ outputs b'
- Output 1 iff $b' = b$

$\text{PRF}_{\mathcal{D},F}(n)$

- Challenger chooses $b \leftarrow \{0, 1\}$
- If $b = 0$: $f \leftarrow \mathcal{F}_n$; oracle $\mathcal{O} = f$
- If $b = 1$: $k \leftarrow \{0, 1\}^n$; oracle $\mathcal{O} = F_k$
- $\mathcal{D}^{\mathcal{O}(\cdot)}$ outputs b'
- Output 1 iff $b' = b$

Definition

F is a secure PRF if, for every PPT $\mathcal{D}$,

$$\Pr[\text{PRF}_{\mathcal{D},F}(n) = 1] \leq \frac{1}{2} + \text{negl}(n).$$

The PRF Security Game

Challenger

Distinguisher $\mathcal{D}$

The PRF Security Game

Challenger

$b \leftarrow \{0, 1\}$

Distinguisher $\mathcal{D}$

The PRF Security Game

Challenger

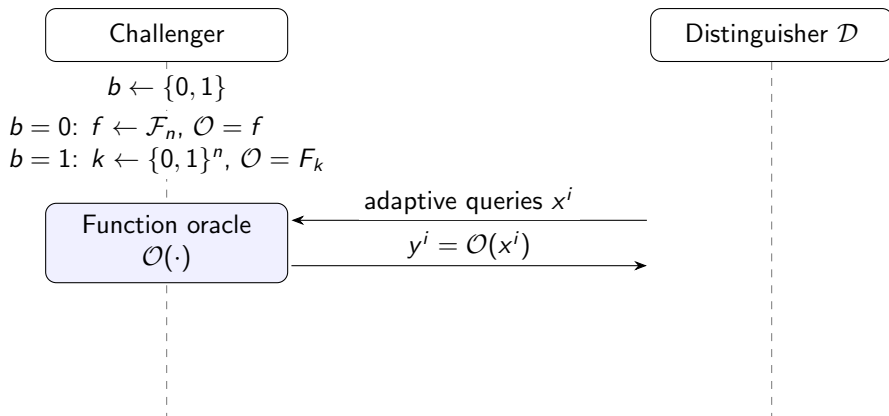
$b \leftarrow \{0, 1\}$

$b = 0: f \leftarrow \mathcal{F}_n, \mathcal{O} = f$

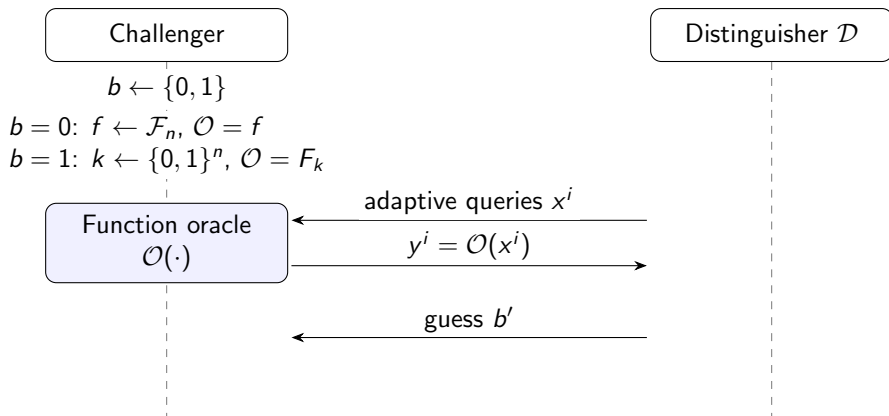
$b = 1: k \leftarrow \{0, 1\}^n, \mathcal{O} = F_k$

Distinguisher $\mathcal{D}$

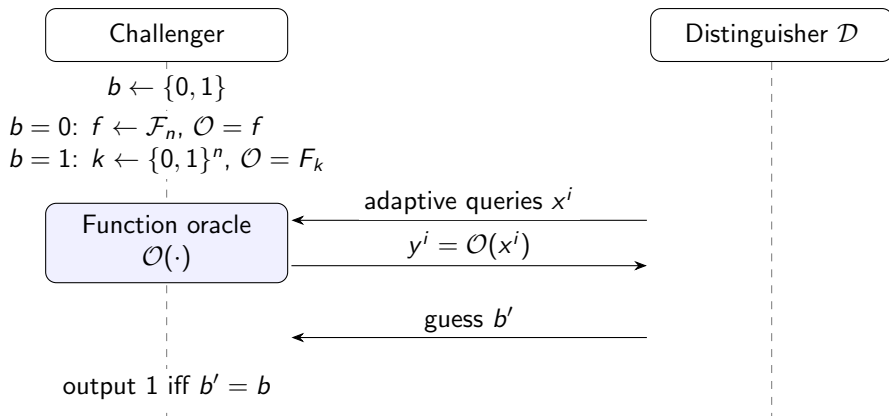
The PRF Security Game



The PRF Security Game



The PRF Security Game



- Polynomially many oracle queries

PRF Observations

- Polynomially many oracle queries
- Adaptive queries

PRF Observations

- Polynomially many oracle queries
- Adaptive queries
- All observed evaluations must look random

PRF Observations

- Polynomially many oracle queries
- Adaptive queries
- All observed evaluations must look random
- Security impossible if $\mathcal{D}$ knows k

- Polynomially many oracle queries
- Adaptive queries
- All observed evaluations must look random
- Security impossible if $\mathcal{D}$ knows k
- Repeated input gives repeated output in both worlds

An Example

Example: Is the following F a secure PRF?

$$F_k(x) = k \oplus x$$

An Example

Example: Is the following F a secure PRF?

$$F_k(x) = k \oplus x$$

The Attack: Choose $x_1 \neq x_2$; query $y_1 = \mathcal{O}(x_1)$ and $y_2 = \mathcal{O}(x_2)$.

An Example

Example: Is the following F a secure PRF?

$$F_k(x) = k \oplus x$$

The Attack: Choose $x_1 \neq x_2$; query $y_1 = \mathcal{O}(x_1)$ and $y_2 = \mathcal{O}(x_2)$.

$$y_1 \oplus y_2 \stackrel{?}{=} x_1 \oplus x_2$$

An Example

Example: Is the following F a secure PRF?

$$F_k(x) = k \oplus x$$

The Attack: Choose $x_1 \neq x_2$; query $y_1 = \mathcal{O}(x_1)$ and $y_2 = \mathcal{O}(x_2)$.

$$y_1 \oplus y_2 \stackrel{?}{=} x_1 \oplus x_2$$

- $\mathcal{O} = F_k$: equality with probability 1

An Example

Example: Is the following F a secure PRF?

$$F_k(x) = k \oplus x$$

The Attack: Choose $x_1 \neq x_2$; query $y_1 = \mathcal{O}(x_1)$ and $y_2 = \mathcal{O}(x_2)$.

$$y_1 \oplus y_2 \stackrel{?}{=} x_1 \oplus x_2$$

- $\mathcal{O} = F_k$: equality with probability 1
- $\mathcal{O} = f$: equality with probability 2^{-n}

An Example

Example: Is the following F a secure PRF?

$$F_k(x) = k \oplus x$$

The Attack: Choose $x_1 \neq x_2$; query $y_1 = \mathcal{O}(x_1)$ and $y_2 = \mathcal{O}(x_2)$.

$$y_1 \oplus y_2 \stackrel{?}{=} x_1 \oplus x_2$$

- $\mathcal{O} = F_k$: equality with probability 1
- $\mathcal{O} = f$: equality with probability 2^{-n}
- Distinguishing success: $1 - 2^{-(n+1)}$

- Pseudorandom permutation (PRP)
 - Each F_k : permutation
 - Compare with a random permutation

Variants of PRFs

- Pseudorandom permutation (PRP)
 - Each F_k : permutation
 - Compare with a random permutation
- Strong PRP
 - Oracle access to F_k and F_k^{-1}
 - Compare with f and f^{-1} for random permutation f

Variants of PRFs

- Pseudorandom permutation (PRP)
 - Each F_k : permutation
 - Compare with a random permutation
- Strong PRP
 - Oracle access to F_k and F_k^{-1}
 - Compare with f and f^{-1} for random permutation f
- Blockcipher: practical PRP / strong PRP

- Same broad goal: efficient pseudorandomness

PRGs and PRFs

- Same broad goal: efficient pseudorandomness
- Different interfaces
 - PRG: one long output
 - PRF: many adaptive input/output queries

PRGs and PRFs

- Same broad goal: efficient pseudorandomness
- Different interfaces
 - PRG: one long output
 - PRF: many adaptive input/output queries
- $\text{PRF} \Rightarrow \text{PRG}$

PRGs and PRFs

- Same broad goal: efficient pseudorandomness
- Different interfaces
 - PRG: one long output
 - PRF: many adaptive input/output queries
- $\text{PRF} \Rightarrow \text{PRG}$
- $\text{PRG} \Rightarrow \text{PRF}$ (with more work)

Quiz time

Outline

- 1 Lecture 2 Review
- 2 Proofs by Reduction
- 3 Chosen-Plaintext Attack Security
- 4 Pseudorandom Functions
- 5 CPA-Secure Encryption from a PRF**

PRF+OTP Encryption

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$

- $\text{Gen}(1^n)$: $k \leftarrow \{0, 1\}^n$
- $\text{Enc}_k(m)$: For $m \in \{0, 1\}^n$, choose $r \leftarrow \{0, 1\}^n$; output

$$c = (r, F_k(r) \oplus m)$$

- $\text{Dec}_k(c)$: Parse $c = (r, c')$ and output $F_k(r) \oplus c'$

PRF+OTP Encryption

$\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$

- $\text{Gen}(1^n)$: $k \leftarrow \{0,1\}^n$
- $\text{Enc}_k(m)$: For $m \in \{0,1\}^n$, choose $r \leftarrow \{0,1\}^n$; output

$$c = (r, F_k(r) \oplus m)$$

- $\text{Dec}_k(c)$: Parse $c = (r, c')$ and output $F_k(r) \oplus c'$
- $F_k(r)$: per-ciphertext “OTP”
- r included for decryption
- Fresh r : fresh-looking mask

Theorem

If F is a secure PRF, then PRF+OTP is CPA-secure.

We will prove this next week.